

UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e de Tecnologia

ARQUITETURAS DE ALTO DESEMPENHO

Relatório da Prática da Aula 2: Implementação de filtros a partir de arquitetura pipeline em FPGA

Docente: Prof. Dr. Emerson Carlos Pedrino

Discentes

Enzo Dezem Alves

RA: 801743

Curso: Engenharia de Computação

Gustavo de Oliveira Gimenes

RA: 820759

Curso: Engenharia de Computação

Guilherme Bartoletti Oliveira

RA: 821881

Curso: Engenharia de Computação

Gustavo Cesar Bento Laurindo

RA: 821402

Curso: Engenharia de Computação

São Carlos, 25 de maio de 2026

1 Introdução

A crescente demanda por aplicações de visão computacional tem exposto os limites das arquiteturas de processamento tradicionais. Na arquitetura clássica de von Neumann, utilizada em processadores de propósito geral (CPUs), a execução de algoritmos de processamento de imagens sofre com o gargalo do barramento de memória, uma vez que cada pixel, ou grupo de pixels, precisa ser buscado, processado sequencialmente e reescrito na memória. Para imagens de alta resolução ou fluxos de vídeo em tempo real, esse paradigma introduz latências inaceitáveis.

Como alternativa a esse modelo, o uso de hardware reconfigurável, como os *Field Programmable Gate Arrays* (FPGAs), tem se consolidado como uma solução de altíssimo desempenho. Os FPGAs permitem a exploração do paralelismo espacial, onde algoritmos são mapeados diretamente em circuitos lógicos (portas e registradores). Isso possibilita que múltiplos dados sejam processados simultaneamente em um único ciclo de *clock*, executando operações complexas à medida que os dados fluem da fonte para o destino (*stream processing*).

O presente relatório detalha a implementação prática de um sistema digital em hardware, desenvolvido na placa DE10 (tecnologia FPGA), projetado para realizar a filtragem morfológica (Máximo e Mínimo) de uma imagem binária ruidosa de 256×128 pixels armazenada em memória, bem como uma filtragem derivativa para detecção de bordas. O objetivo principal do projeto abrangeu não apenas a construção do filtro espacial em linguagem de descrição de hardware (Verilog/VHDL), mas também a sua integração com um controlador de vídeo VGA. O sistema permite a visualização em tempo real do processamento, fornecendo chaves de controle para alternar dinamicamente entre a imagem original e a filtrada, além de implementar o mascaramento correto para posicionar a imagem no monitor.

2 Desenvolvimento Teórico

2.1 Dispositivos lógicos programáveis e paralelismo espacial

Conforme abordado na disciplina, os PLDs (Dispositivos Lógicos Programáveis) evoluíram significativamente, culminando nos atuais FPGAs. Estes CIs são compostos por uma vasta matriz de blocos lógicos configuráveis interligados por uma rede de roteamento programável. Em vez de executar instruções estáticas baseadas em um *Program Counter*, o FPGA sintetiza o próprio "caminho de dados" (*datapath*). Para aplicações de vídeo, isso significa que enquanto a CPU faria laços de repetição (*loops* aninhados) para varrer os eixos *X* e *Y* de uma imagem, o FPGA cria um duto lógico contínuo (um *pipeline*) por onde os pixels passam e saem já filtrados, sem sobrecarga (*overhead*) de busca de instruções.

2.2 Morfologia matemática: filtros de Máximo e Mínimo

No domínio do processamento digital de imagens, operações morfológicas são técnicas não-lineares fundamentadas na teoria dos conjuntos, utilizadas para extrair componentes da imagem que sejam úteis para a representação e descrição da forma das regiões. Em imagens binárias (1 bit por pixel), os filtros propostos operam da seguinte maneira:

- **Filtro de Mínimo (Erosão):** O valor de saída do pixel central de uma janela (ex: 3×3) será o valor mínimo entre todos os vizinhos. Tratando-se de lógica digital binária, o mínimo só será 1 se **todos** os pixels da janela forem 1. Portanto, em hardware, a erosão é modelada por uma porta lógica **AND** de múltiplas entradas. Ela é responsável por "corroer" os limites dos objetos brancos, eliminando efetivamente pontos ruidosos brancos isolados em fundos pretos.
- **Filtro de Máximo (Dilatação):** O valor de saída do pixel central será o valor máximo da vizinhança. O pixel será 1 se **pelo menos um** vizinho for 1. Em hardware, isso é mapeado como uma grande porta **OR**. A dilatação expande os limites dos objetos brancos, preenchendo pequenos "buracos" ou falhas (ruídos pretos isolados dentro de áreas brancas).

2.3 Detecção de bordas

A detecção de bordas consiste em identificar regiões da imagem onde ocorre uma variação significativa de intensidade entre pixels vizinhos. Matematicamente, isso é modelado através de operadores derivativos, que respondem fortemente a mudanças abruptas no sinal.

Em imagens discretas, a derivada é aproximada por diferenças finitas. O gradiente da imagem pode ser decomposto nas direções horizontal e vertical:

$$G_x(x, y), \quad G_y(x, y)$$

A magnitude do gradiente pode ser estimada por:

$$|G(x, y)| = |G_x(x, y)| + |G_y(x, y)|$$

ou, de forma mais precisa:

$$|G(x, y)| = \sqrt{G_x^2 + G_y^2}$$

Filtros clássicos como Sobel utilizam máscaras convolucionais 3×3 que combinam derivação com suavização, tornando a detecção de bordas mais robusta a ruídos. As máscaras do operador de Sobel são dadas por:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

No contexto deste trabalho, foi implementada uma versão simplificada do operador de Sobel em hardware. As multiplicações pelos coeficientes foram substituídas por operações de deslocamento (shift), reduzindo significativamente o custo computacional.

Além disso, a magnitude do gradiente foi aproximada pela soma dos valores absolutos:

$$|G(x, y)| \approx |G_x| + |G_y|$$

Essa abordagem permite uma implementação eficiente em FPGA, mantendo um bom compromisso entre qualidade da detecção de bordas e utilização de recursos, viabilizando o processamento em tempo real dentro do pipeline de vídeo.

2.4 Interface de vídeo VGA

O *Video Graphics Array* (VGA) é um padrão que exige controle temporal rigoroso. O feixe eletrônico do monitor (ou sua matriz digital de varredura) atualiza a tela linha a linha, da esquerda para a direita, de cima para baixo. Os sinais de sincronismo horizontal (*hsync*) e vertical (*vsync*) dizem ao monitor quando retornar à margem esquerda e ao topo da tela, respectivamente. Entre as áreas ativas de imagem existem os intervalos de "escuridão" (*blanking*, *front porch* e *back porch*). O FPGA precisa enviar os valores de cor (RGB) estritamente durante o período ativo; qualquer atraso computacional corrompe a estabilidade da imagem.

3 Desenvolvimento Prático

A implementação prática do sistema demandou a integração de um módulo de processamento de imagens personalizado à infraestrutura de vídeo e controle pré-existente fornecida pelo docente. O sistema-base contemplava a configuração do PLL (gerador de *clock*), a interface de sincronismo VGA (*VGA_Sync*), a inicialização da memória ROM a partir do arquivo *.mif* da imagem e a definição da pinagem (*.qsf*) para a placa DE10. O escopo principal do projeto concentrou-se na modelagem e inserção do filtro espacial de hardware.

3.1 Concatenação de endereço e janelamento

Para armazenar a imagem a ser exibida, foi utilizada uma memória do tipo ROM, responsável por fornecer o valor do pixel correspondente à posição atual de varredura do VGA. Para isso,

desenvolveu-se um módulo de concatenação que combina os sinais de linha e coluna do pixel corrente (pixel_row e pixel_column), formando o endereço de acesso à memória.

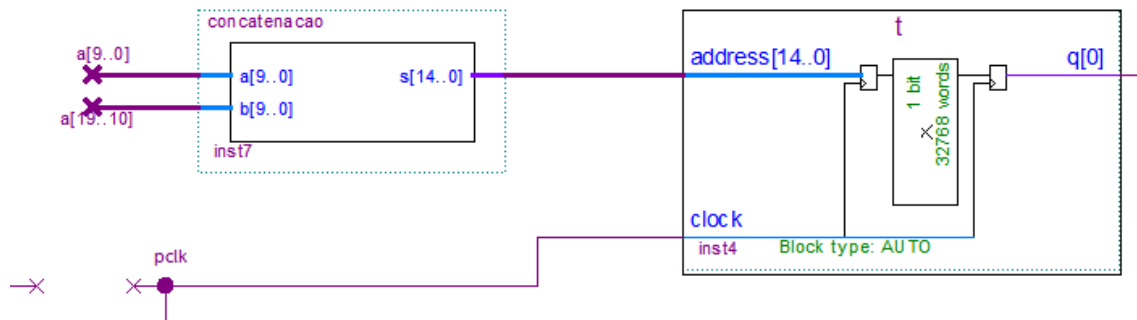


Figura 1: Bloco esquemático do módulo de concatenação.

```

1 module concatenacao(a,b,s);
2     input [9:0] a;
3     input [9:0] b;
4     output [14:0] s;
5
6     assign s = { b[6:0],a[7:0]};
7 endmodule

```

Devido à limitação de memória, apenas subconjuntos dos bits de linha e coluna foram utilizados na concatenação, resultando em um endereço de 15 bits compatível com a ROM implementada.

Dessa forma, conhecendo-se o pixel que está sendo varrido pelo controlador VGA, é possível utilizá-lo diretamente como endereço para leitura da ROM, obtendo o valor do pixel correspondente da imagem armazenada.

Como a capacidade de memória da FPGA é limitada, optou-se por utilizar uma imagem de resolução reduzida (256x128 pixels), exibida no canto superior esquerdo da tela. Para garantir que apenas essa região seja apresentada, foi implementado um módulo de mascaramento, que força a saída para zero quando a varredura estiver fora da região de interesse. Isso evita a exibição de dados inválidos e mantém a consistência visual da imagem.

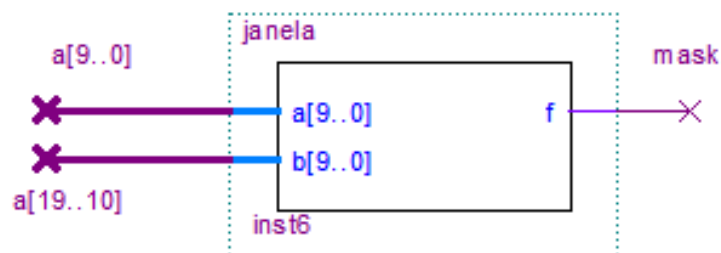


Figura 2: Bloco esquemático do módulo de janelamento.

```

1 module janela(a,b,f);
2     input [9:0] a;
3     input [9:0] b;
4     output f;
5     reg f;
6
7     always@(a,b) begin
8
9         if(a > 'd256 | b > 'd128)
10             begin
11                 f = 0;
12             end
13         else
14             begin
15                 f = 1;
16             end
17         end
18     endmodule

```

3.2 Arquitetura do estágio de filtragem

O desafio central do processamento espacial de vídeo em tempo real é a necessidade de avaliar não apenas o pixel em curso, mas também seus arredores imediatos de forma simultânea. Para viabilizar a aplicação da máscara morfológica sobre uma vizinhança de 8 conexões (janela 3×3), utilizou-se uma arquitetura baseada em registradores de deslocamento (*Shift Registers*) distribuídos em múltiplas linhas.

A solução implementada pelo grupo está apresentada abaixo:

```

1 module filtro (
2     input clk, img, sel, valid,
3     output img_out, valid_out
4 );
5
6     reg [255:0] reg0;
7     reg [255:0] reg1;
8     reg [2:0] reg2;
9     reg v_out;
10
11     wire [8:0] window;
12
13     always @(posedge clk) begin
14         if (valid) begin
15             reg0 <= {reg0[254:0], reg1[255]};
16             reg1 <= {reg1[254:0], img};
17             reg2 <= {reg2[1:0], reg0[255]};

```

```

18         v_out <= valid;
19     end
20 end
21
22     assign window = {reg0[2:0], reg1[2:0], reg2};
23     assign valid_out = valid;
24     assign img_out = valid ? (sel ? &window:|window) : 1'b0;
25 endmodule

```

No escopo interno do módulo, foram utilizados três buffers principais (`reg0`, `reg1` e `reg2`), que armazenam pixels de linhas consecutivas da imagem. Os registradores `reg0` e `reg1` possuem largura de 256 bits, funcionando como linhas de atraso, enquanto `reg2` armazena os pixels mais recentes. Essa organização permite reconstruir, a cada ciclo de clock, a vizinhança vertical necessária para a janela 3×3 .

Em um bloco processual síncrono sensível à borda de subida do *clock*, os dados são atualizados de forma controlada por meio do sinal de validade (`valid`). O mecanismo de deslocamento ocorre tanto horizontal quanto verticalmente: os buffers superiores recebem dados deslocados de suas posições anteriores (`reg0 <= {reg0[254:0], reg1[255]}`), enquanto o buffer intermediário incorpora o novo pixel de entrada (`img`). Esse encadeamento implementa um pipeline contínuo, no qual pixels antigos são descartados e novos pixels são inseridos a cada ciclo válido.

O uso do sinal `valid` é fundamental para garantir a coerência temporal do sistema. Notou-se empiricamente que, durante o processamento da imagem e shifts dos buffers, existem intervalos em que os dados de entrada não correspondiam exatamente a pixels visíveis da imagem. Caso os registradores fossem atualizados continuamente, esses valores inválidos seriam inseridos no pipeline, comprometendo o alinhamento entre as linhas armazenadas e resultando em artefatos visuais, como tremulação da imagem. Ao condicionar a atualização dos buffers ao sinal `valid`, assegura-se que apenas pixels válidos participem da formação da vizinhança, preservando a consistência do processamento.

A janela de processamento é então formada de maneira combinacional por meio da concatenação de subconjuntos dos três buffers: `window = {reg0[2:0], reg1[2:0], reg2}`. Dessa forma, obtém-se simultaneamente o pixel central e seus oito vizinhos imediatos, sem interrupção do fluxo de dados. O sinal `valid` também é propagado para a saída (`valid_out`), garantindo que apenas dados corretamente sincronizados sejam encaminhados para as etapas seguintes do sistema.

3.3 Lógica de filtragem: Mínimo e Máximo

Com os 9 pixels da vizinhança mantidos estáveis durante o ciclo de relógio atual através dos flip-flops, a filtragem foi resolvida em uma única etapa combinacional, minimizando a latência do *datapath*.

A sequência de filtros morfológicos adotada no projeto não foi arbitrária, mas sim definida com o objetivo de melhorar a qualidade da imagem binária sob diferentes tipos de ruído.

Inicialmente, aplica-se um filtro de mínimo seguido de um filtro de máximo, caracterizando uma operação conhecida como *abertura* (opening). Essa etapa é responsável por remover ruídos brancos isolados, preservando a estrutura principal dos objetos na imagem.

Em seguida, aplica-se uma nova sequência composta por um filtro de máximo seguido de um filtro de mínimo, configurando uma operação de *fechamento* (closing). Essa etapa tem como objetivo preencher pequenas falhas e buracos em regiões brancas, além de suavizar contornos.

Portanto, a sequência completa implementada:

mínimo $\rightarrow$ máximo $\rightarrow$ máximo $\rightarrow$ mínimo

corresponde à aplicação combinada de abertura e fechamento, permitindo simultaneamente a remoção de ruídos isolados e a recuperação da integridade das regiões da imagem.

Essa estratégia mostrou-se eficaz para o tipo de imagem binária utilizada, proporcionando um equilíbrio entre remoção de ruído e preservação de estruturas relevantes.

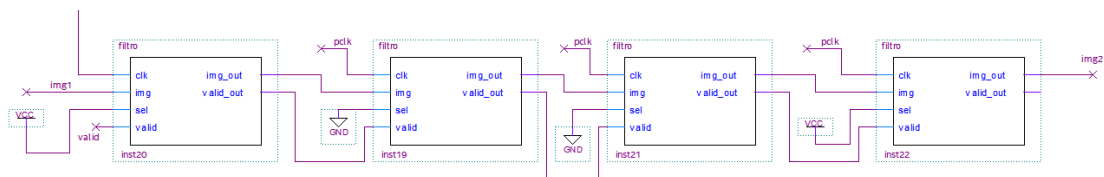


Figura 3: Pipeline de filtragem.

3.4 Implementação do filtro derivativo (Sobel)

Além das operações morfológicas, foi implementado um filtro derivativo com o objetivo de realizar a detecção de bordas em tempo real. Diferentemente da filtragem morfológica, que depende de operações lógicas sobre a vizinhança, o filtro derivativo baseia-se na estimação do gradiente espacial da imagem.

A implementação foi realizada a partir da mesma arquitetura de extração de vizinhança descrita anteriormente, reutilizando a janela 3×3 formada pelos sinais $w0$ até $w8$. Isso permite que o cálculo das derivadas horizontais e verticais seja feito de forma totalmente combinacional a cada ciclo de clock, sem necessidade de estágios adicionais de armazenamento.

O operador implementado corresponde a uma versão simplificada do filtro de Sobel, cujas máscaras são dadas por:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Na implementação em hardware, as multiplicações pelos coeficientes foram substituídas por operações de deslocamento (*shift*), explorando o fato de que os pesos do filtro são potências de dois. Por exemplo, coeficientes iguais a 2 são implementados por deslocamento à esquerda de um bit, reduzindo significativamente o uso de recursos lógicos.

Os gradientes nas direções horizontal (*gx*) e vertical (*gy*) são obtidos pela diferença entre somatórios de termos positivos e negativos da vizinhança. Em seguida, calcula-se o valor absoluto de cada componente utilizando complemento de dois, conforme implementado no código:

```
1 assign gx_abs = gx[3] ? !gx+1'b1 : gx;
2 assign gy_abs = gy[3] ? !gy+1'b1 : gy;
```

A magnitude do gradiente é então aproximada pela soma dos módulos:

```
1 assign magnitude = gx_abs + gy_abs;
```

Essa aproximação elimina a necessidade de operações de raiz quadrada, que seriam custosas em hardware, mantendo ainda uma boa qualidade na detecção de bordas.

Por fim, a saída do filtro é obtida por meio de uma operação de limiarização (*thresholding*), na qual apenas valores de magnitude acima de um determinado nível são considerados bordas:

```
1 assign pixel = (magnitude >= T) ? 1'b1 : 1'b0;
2 assign img_out = valid ? pixel : 1'b0;
```

onde T é um limiar fixo (no caso implementado, $T = 2$). Como resultado, pixels pertencentes a regiões de alta variação de intensidade são destacados em branco, enquanto regiões homogêneas permanecem em preto.

Essa abordagem permite a detecção de bordas em tempo real com baixa complexidade de hardware, integrando-se de forma eficiente ao pipeline de vídeo previamente desenvolvido.

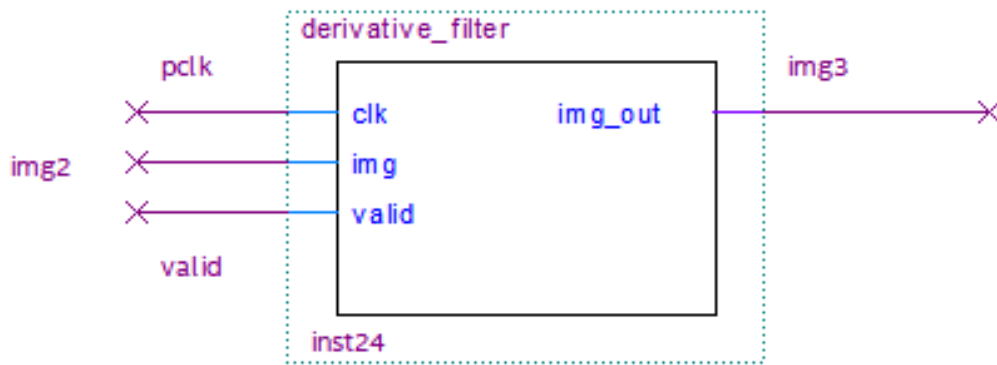


Figura 4: Bloco esquemático do filtro derivativo Sobel.

3.5 Integração dos filtros na visualização

Para permitir a análise comparativa dos diferentes métodos de processamento implementados, foi desenvolvida uma estratégia de integração dos filtros diretamente no caminho de visualização do sistema. Essa abordagem possibilita a seleção dinâmica entre múltiplas versões da imagem processada em tempo real.

A arquitetura de visualização utiliza dois multiplexadores em cascata, responsáveis por selecionar entre diferentes fontes de dados ao longo do *datapath*. No primeiro estágio, um multiplexador controla a escolha entre a imagem original proveniente da memória (*img1*) e uma versão alternativa (*img2*), que é a saída do pipeline de filtros implementados.

No segundo estágio, um novo multiplexador permite selecionar entre a saída do estágio anterior e uma terceira fonte de dados (*img3*), associada ao filtro derivativo. Os sinais de seleção (*sw[0]* e *sw[1]*) são mapeados para chaves físicas da placa, permitindo ao usuário alternar interativamente entre as diferentes representações da imagem.

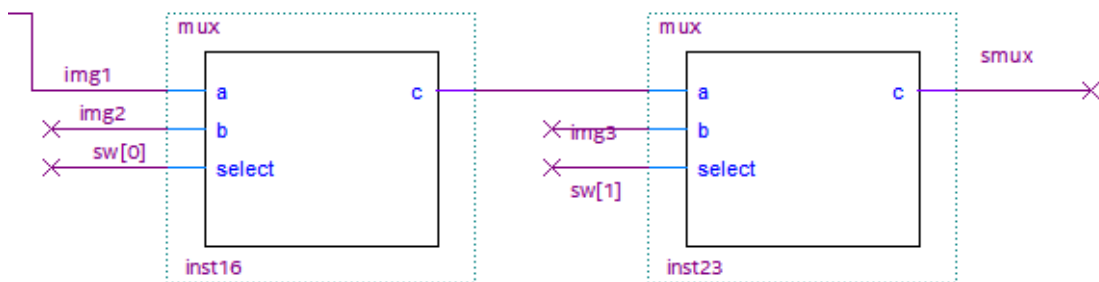


Figura 5: Multiplexadores de seleção da imagem mostrada no VGA.

3.6 Integração no diagrama esquemático (Top-Level)

A etapa conclusiva do projeto consistiu na integração dos módulos no nível de topo, compondo o fluxo completo de processamento de vídeo.

O *datapath* inicia-se na memória ROM, cuja saída (*q*) alimenta uma cadeia de blocos de filtragem conectados em pipeline, permitindo a aplicação sucessiva de operações morfológicas. O sinal de validade (*valid*) é propagado ao longo desses estágios, garantindo a sincronização com o fluxo de pixels.

Adicionalmente, foi incorporado um módulo de detecção de bordas baseado em operador derivativo, operando sobre a saída dos filtros morfológicos.

Para fins de validação, multiplexadores foram inseridos no caminho de dados, permitindo seleccionar entre diferentes estágios do processamento (imagem original, filtrada ou derivada) por meio de chaves da placa. O sinal seleccionado é então encaminhado ao módulo de interface VGA, responsável pela geração dos sinais de sincronismo e pela saída RGB.

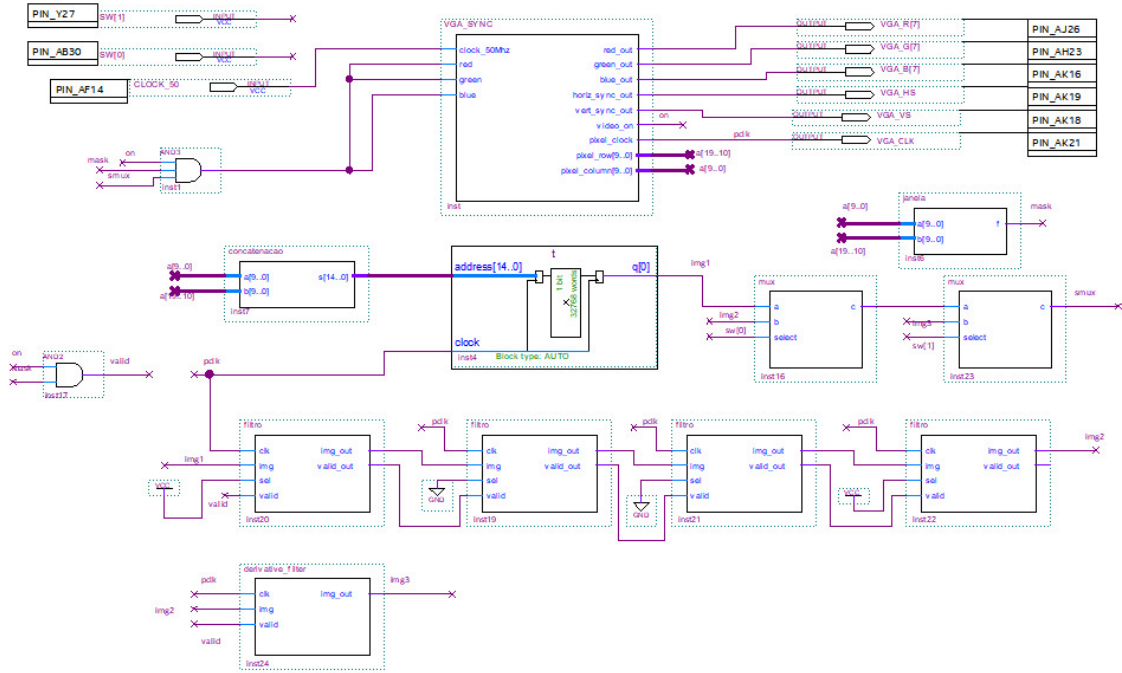


Figura 6: Diagrama esquemático de topo (Top-Level) evidenciando a integração do módulo de filtragem ao caminho de dados do vídeo.

4 Resultados e Discussão

A validação experimental do sistema na placa FPGA DE10 demonstrou que a arquitetura proposta atendeu a todos os requisitos funcionais estabelecidos no roteiro da prática. O primeiro ponto de sucesso foi o **módulo de mascaramento**: como a resolução nativa do controlador VGA é de 640×480 e a imagem de entrada possui apenas 256×128 , a lógica de controle de coordenadas implementada garantiu que o feixe de varredura

fosse "desligado"(RGB forçado a zero) fora da região de interesse. Isso resultou em uma visualização limpa e centralizada no canto superior esquerdo, sem artefatos de borda ou repetições indevidas.

Quanto à **filtragem morfológica**, o sistema foi testado sob as duas perspectivas cobradas:

- **Filtro de Máximo:** Obteve-se êxito na remoção de ruídos escuros internos a objetos brancos, preenchendo falhas de continuidade.
- **Filtro de Mínimo:** Eliminou-se efetivamente os pontos ruidosos isolados no fundo da imagem, "erodindo" artefatos que não pertenciam à estrutura principal.



Figura 7: Foto do monitor com a chave ligada durante execução do processamento com filtros de mínimo e máximo.

Quanto à **detecção de borda**, o sistema apresentou resultados satisfatórios, mas com certa perda de informação em alguns pontos, apresentando linhas pontilhadas e perda do lado superior do retângulo. Acredita-se que houve algum erro de implementação no filtro, problema que não pôde ser devidamente resolvido em razão de falta de tempo para execução da prática.

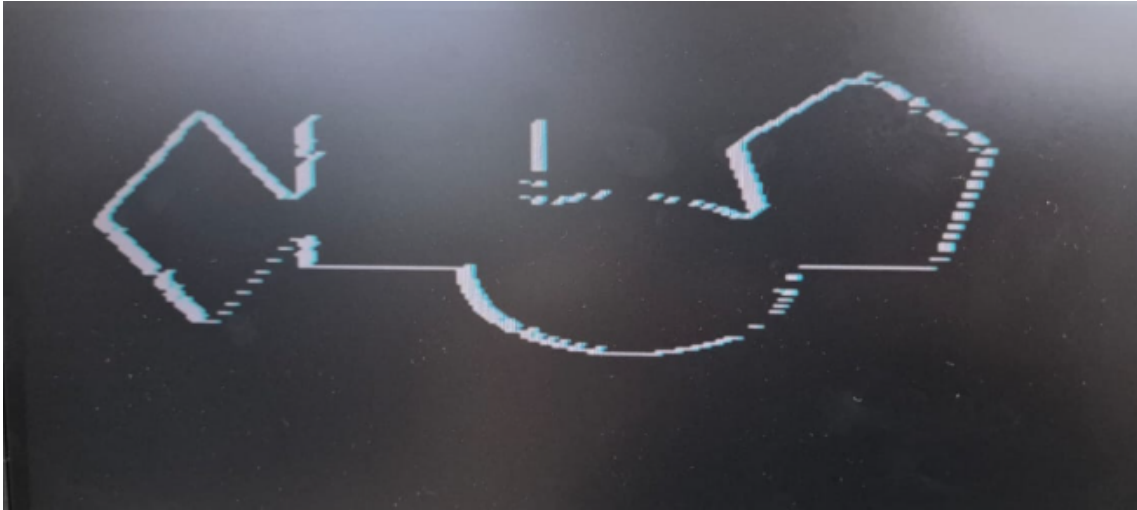


Figura 8: Foto do monitor com a chave ligada durante execução do processamento com filtro derivativo.

Um ponto crítico discutido pelo grupo foi o desempenho em tempo real. Graças ao paralelismo do FPGA, a troca entre a imagem ruidosa (original da ROM) e a imagem processada — realizada via chave seletora física (*switch*) — ocorre de forma instantânea. Não houve degradação relevante do sinal de vídeo ou perda de sincronismo vertical, comprovando que o *datapath* de 1 bit é extremamente eficiente para este tipo de aplicação.

4.1 Análise dos Problemas Técnicos e Depuração de Sintaxe

A maior parte do tempo de desenvolvimento foi consumida na resolução de problemas de sintaxe específicos do Verilog, que impactaram diretamente a lógica de vizinhança de 8 pixels exigida. O principal obstáculo foi a ordem de concatenação nos registradores de deslocamento.

Diferente de linguagens procedurais, o Verilog interpreta a concatenação `{ }` como um mapeamento físico de barramentos. Inicialmente, o grupo enfrentou dificuldades ao inverter a ordem dos bits na atribuição `linha <= {linha[1:0], data_in}`. Esse erro de lógica espacial fazia com que a janela 3×3 "enxergasse" os vizinhos de forma espelhada ou deslocada em um ciclo de clock, invalidando o cálculo de máximo e mínimo. A depuração exigiu uma análise minuciosa do diagrama esquemático e das formas de onda, evidenciando que em hardware, a sintaxe define a topologia de fiação, e qualquer inversão mínima rompe a integridade do filtro espacial.

5 Conclusão

A conclusão desta prática permitiu a verificação integral dos objetivos propostos no roteiro original. Foi possível entender a fundo o funcionamento do controlador VGA e a

necessidade de sincronismo rigoroso entre a leitura da memória ROM e a varredura do monitor. A implementação do filtro de 8 vizinhos em Verilog demonstrou ser uma solução de alto desempenho para o processamento de imagens ruidosas, provando que operações morfológicas binárias podem ser reduzidas a portas lógicas simples (AND/OR) quando a arquitetura de suporte (Line Buffers) é bem desenhada.

Todos os itens cobrados no PDF da prática foram discutidos e implementados:

1. **Leitura de memória:** Executada via arquivo `.mif` em sincronia com os contadores VGA.
2. **Filtro de Máximo/Mínimo:** Implementado com janela de 8 vizinhos (matriz 3×3).
3. **Mascaramento:** Restrição da exibição à área de 256×128 pixels.
4. **Seleção de Visualização:** Implementada via multiplexação controlada por *switch*.
5. **Detector de Bordas:** Implementada via filtro derivativo (Sobel).

Apesar dos contratempos sintáticos com a concatenação de bits e a ordem dos registradores, o projeto final demonstrou-se robusto. A experiência ressaltou que, no contexto de Arquiteturas de Alto Desempenho, o tempo gasto na correta descrição do hardware é compensado pela execução imediata e paralela do algoritmo em tempo real, algo inviável em processadores convencionais para fluxos de vídeo de alta taxa de atualização.

6 Referências

1. PEDRINO, E. C. Sistemas Digitais - Aula 01 (Introdução a PLDs). UFSCar, 2022.
2. PRÁTICA 5. Processamento de Imagens Binárias em Hardware. Roteiro da disciplina Arquiteturas de Alto Desempenho, UFSCar, 2026.
3. Embarcados: Controlador VGA - Parte 1. Disponível em: <https://embarcados.com.br/controlador-vga-parte-1/>.
4. Embarcados: Controlador VGA - Parte 2. Disponível em: <https://embarcados.com.br/controlador-vga-parte-2/>.